Parallel and Distributed Systems

Chapter 6: Synchronization Mechanisms

Prof. Dr. Martin Sträßer

Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Thread Components

- Notably, threads within the same process share large parts of the process memory
- However, threads have:
  - Their own stack (for function calls and local variables)
  - Own signal handling (they are signal disposition, but signals are received by individual threads)
  - Own kernel entries (they share process information, but have also individual data)
  - Their own program counter and register states

Process Component	Shared by Threads
Program code	Yes
Stack	No
Heap	Yes
Environment variables	Yes
Open file descriptors	Yes
Signal handlers	Partially
Current working directory	Yes
Kernel metadata	Partially

This makes creating threads lightweight! There is not many data that has to be copied!

Recap: Challenges with Threads

- Read and write access to shared resources can lead to unpredictable behavior

```python
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

```python
if __name__ == '__main__':
    num_threads = 20
    n = 100
    threads = []
    for i in range(num_threads):
        t = threading.Thread(target=increment_worker, args=(f'{i+1}', n))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    print(f'Expected counter: {num_threads * n}')
    print(f'Actual counter:   {counter}')
```

Overview

- Critical Sections, Race Conditions, Atomic Operations
- Locks
- Reentrant Locks
- Condition Variables
- Barriers
- Semaphores
- Events
- Queues for Thread Communication
- Lock-Free Data Structures
- Process Safety and Thread Safety

Critical Sections, Race Conditions, Atomic Operations

Critical Section

- Part of program code where shared resources (memory) is accessed reading **and** writing
- If multiple concurrent accesses to the shared resource happen in this critical section, the results are unpredictable
- Mutual exclusion = only one thread/process can be inside the critical section at the same time

```python
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):  1 usage
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

Race Conditions

- Without mutual exclusion, results depend on scheduling decisions of the operating system
- This situation is a **race condition**
- Race conditions
  - Cause different symptoms in different executions of the same program code
  - Are very hard to debug

```python
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):  1 usage
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

1
2
3

- Example
  - Two threads A and B
  - Execution order: A1 – A2 – A3 – B1 – B2 – B3, Result: counter = 2
  - Execution order: A1 – B1 – A2 – B2 – A3 – B3, Result: counter = 1

Atomic Operations

- A critical role in concurrent or parallel programming is played by **atomic operations**
- Atomic operations are indivisible
- They either complete or fail/do not happen
- Other threads/processes can not see intermediate states in the operation
- Core problem in modern operating systems: context switches happen very often (e.g., caused by high-priority processes or time slicing)
- Consequence: Atomicity can only be achieved for very small operations (few CPU instructions, low runtime)

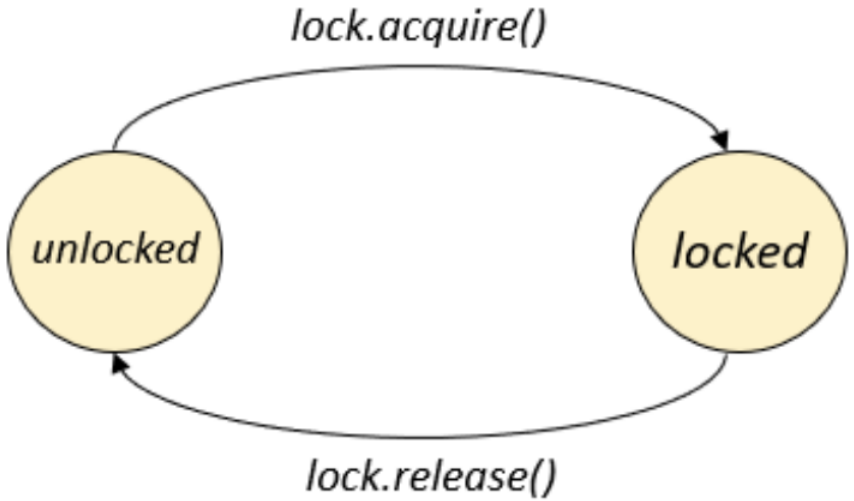
Challenge

- Critical sections can involve quite costly tasks (e.g., reading and writing to a file)
- If they cannot be executed atomically, how to ensure mutual exclusion?
- Conceptual solution: Block concurrent threads/processes (make them not runnable)
- Any other context switch happening in a critical section will have no effect on shared resource

Locks

Locks

- Simplest synchronization mechanism
- When entering a critical section, lock has to be acquired
- When leaving a critical section, lock is released
- Lock can be acquired by one thread at a time
- Acquire and release operations are atomic
- Native support by operating system



Example: Locks

```python
import threading
import time

counter = 0
counter2 = 0
lock = threading.Lock()

def increment_sync():  1 usage
    global counter
    for _ in range(1000):
        lock.acquire()
        temp = counter
        time.sleep(0.000001)
        counter = temp + 1
        lock.release()

def increment():  1 usage
    global counter2
    for _ in range(1000):
        temp = counter2
        time.sleep(0.000001)
        counter2 = temp + 1
```

```python
if __name__ == '__main__':
    threads = [threading.Thread(target=increment_sync) for _ in range(4)]
    start = time.perf_counter()

    for t in threads:
        t.start()
    for t in threads:
        t.join()

    end = time.perf_counter()
    print(counter)
    print(f"Elapsed: {end - start:.6f} seconds")

    threads = [threading.Thread(target=increment) for _ in range(4)]

    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    print(counter2)
    end = time.perf_counter()
    print(f"Elapsed: {end - start:.6f} seconds")
```

```
4000
Elapsed: 1.059134 seconds
1005
Elapsed: 0.534999 seconds
```

See code no. 1

Deadlocks

- Acquire operation is blocking
- Holding one lock and trying to acquire another lock can lead to deadlocks
- Also: It has to be guaranteed that a lock is released at some time
- Scenario
  - Lock is a global object accessible by multiple threads
  - Thread A acquires lock
  - Thread A is forcefully terminated
  - Lock still exists and never gets released

Example: Deadlocks

```python
import threading
import time

lock_a = threading.Lock()
lock_b = threading.Lock()

def thread_1():  1 usage
    print("Thread 1: acquiring lock A")
    lock_a.acquire()
    time.sleep(0.1)
    print("Thread 1: acquiring lock B")
    lock_b.acquire()
    print("Thread 1: acquired both locks")
```

```python
def thread_2():  1 usage
    print("Thread 2: acquiring lock B")
    lock_b.acquire()
    time.sleep(0.1)
    print("Thread 2: acquiring lock A")
    lock_a.acquire()
    print("Thread 2: acquired both locks")

if __name__ == '__main__':
    t1 = threading.Thread(target=thread_1)
    t2 = threading.Thread(target=thread_2)
    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

See code no. 2

Python Locks with Context Manager

- In Python, the with statement creates a runtime context that allows you to execute a block of statements under the control of a context manager
- The with expression must return an object that implements a context management protocol which essentially consists of two methods: __enter__() and __exit__()
- In addition to this, with statement has the further advantage of incorporating the functionality of the try ... finally construct
- In the case of locks, the acquire() method will be called when entering the block and the release() method will be called when exiting
- Also supported for Rlock, Condition, and Semaphore

Python Locks with Context Manager

```python
import threading
import time

counter = 0
counter2 = 0
lock = threading.Lock()

def increment():  1 usage
    global counter
    for _ in range(1000):
        lock.acquire()
        try:
            temp = counter
            time.sleep(0.000001)
            counter = temp + 1
        finally:
            lock.release()
```

```python
def increment_context():  1 usage
    global counter2
    for _ in range(1000):
        with lock:
            temp = counter2
            time.sleep(0.000001)
            counter2 = temp + 1

if __name__ == '__main__':
    threads = [threading.Thread(target=increment) for _ in range(4)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    print(counter)
    threads = [threading.Thread(target=increment_context) for _ in range(4)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    print(counter2)
```

See code no. 3

Asymmetric Use of Locks

```python
import threading
import time

shared = 0
lock = threading.Lock()

def funcA():  1 usage
    global shared
    for i in range(10):
        time.sleep(1)
        shared += 10
        print(f"Thread A wrote: {shared}")
        lock.acquire()
```

```python
def funcB():  1 usage
    global shared
    lock.acquire()
    for i in range(10):
        time.sleep(1)
        shared -= 10
        print(f"Thread B wrote: {shared}")
        lock.release()

if __name__ == '__main__':
    t1 = threading.Thread(target=funcA)
    t2 = threading.Thread(target=funcB)
    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

- What will be the output here?

See code no. 4

Identity of Threads in Locks

- A lock does not know the identity of a thread/does not know its owner
- Another thread can release it
- Simple locks do not have ownership tracking in most programming languages (exception: synchronized keyword in Java)

Reentrant Locks

Reentrant Locks

- Reentrant locks can be acquired and released multiple times by the same thread
- They hold information on the owner thread and on the recursion level
- Unlike normal locks, the pairs of calls to the acquire() and release() methods are multiple, and can be nested together
- To fully unlock, the same number of release operations and acquire operations before has to be executed
- Good when critical sections are spread across different methods/functions or classes

Reentrant Locks

```python
import threading
import time
from threading import Lock, RLock

# lock = Lock()
lock = RLock()

def outer(name: str):  1 usage
    print(f"{name} wants lock")
    with lock:
        print(f"{name} in lock")
        time.sleep(1)
        inner(name)
        print("Outer output")
    print(f"{name} released lock")
```

```python
def inner(name: str):  1 usage
    print(f"{name} wants lock")
    with lock:
        print(f"{name} in lock")
        print("Inner output")
    print(f"{name} released lock")

if __name__ == '__main__':
    threads = [threading.Thread(target=outer, args=(f"Thread-{i}",)) for i in range(2)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

See code no. 5

Condition Variables

Condition Variables

- A condition variable identifies a change of state in the application
- This is a synchronization mechanism where a thread waits for a specific condition and another thread notifies that this condition has taken place
- Once the condition takes place, the thread acquires the lock to get exclusive access to the shared resource

Condition Variables

- The Condition class has an internal Lock, which with acquire() and release() passes through the locked and unlocked states
- But, the Condition class has also other methods
  - The wait() method releases the lock but blocks the thread until another thread calls the notify() and notify_all() methods
  - The notify() method wakes up only one of the threads waiting for the condition variable if any
  - The notify_all() method wakes up all waiting threads

Method	Explanation
acquire()	Acquire the lock
wait()	Release lock and sleep (you must hold the lock)
notify()	Wake next waiting thread (you must hold the lock)
notify_all()	Wake all waiting threads (you must hold the lock)
release()	Release the lock

Condition Variables

```python
import threading
import time

condition = threading.Condition()
queue = []
MAX_ITEMS = 5


def producer():  1 usage
    for i in range(10):
        with condition:
            while len(queue) == MAX_ITEMS:
                condition.wait()
            queue.append(i)
            print(f"Produced {i}")
            condition.notify_all()
        time.sleep(0.5)
```

```python
def consumer():  1 usage
    for _ in range(10):
        with condition:
            while len(queue) == 0:
                condition.wait()
            item = queue.pop(0)
            print(f"Consumed {item}")
            condition.notify_all()
        time.sleep(1)

if __name__ == '__main__':
    t1 = threading.Thread(target=producer)
    t2 = threading.Thread(target=consumer)
    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

See code no. 6

Barriers

Barriers

- Barriers allow synchronization at a checkpoint
- All threads have to reach a specific instruction
- Afterward, all threads are allowed to continue
- Analogy: We are going for a hike, everyone has their own speed, but we all have to meet at the car pick-up at the end

Barriers: Methods

Methods	Explanation
Constructor: Barrier(n)	Creates a barrier where n threads have to meet
Constructor: Barrier(n, action=func)	Creates a barrier where n threads have to meet, executes action when barrier is passed
wait()	Thread waits at barrier (blocking operation), released when n-th thread arrives
wait(timeout=t)	Thread waits at barrier at most t seconds, BrokenBarrierError raised if timeout
reset()	Break the barrier and reset count to 0, every waiting thread is woke up and raises BrokenBarrierError
abort()	Break the barrier permanently, every thread is woke up and raises BrokenBarrierError

Barriers

```python
import time
from threading import Thread, Barrier
import random

def on_barrier_release():  1 usage
    print("Barrier released")

barrier = Barrier( parties: 4, action=on_barrier_release)
```

```python
def phase_worker(i):  1 usage
    print(f"Worker {i}: phase 1")
    time.sleep(random.randint( a: 1,  b: 5))
    barrier.wait()

    print(f"Worker {i}: phase 2")
    time.sleep(random.randint( a: 1,  b: 5))
    barrier.wait()

    print(f"Worker {i}: phase 3")
    time.sleep(random.randint( a: 1,  b: 5))
    barrier.wait()

if __name__ == '__main__':
    threads = [Thread(target=phase_worker, args=(i,)) for i in range(4)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

See code no. 7

Broken Barriers

```python
import threading
import time
import random

NUM_WORKERS = 4
barrier = threading.Barrier(NUM_WORKERS)

def worker(worker_id):  1 usage
    try:
        print(f"Worker {worker_id}: initializing...")
        time.sleep(random.randint( a: 1,  b: 5))

        # Simulate a failure in one worker
        if worker_id == 2:
            raise ValueError("Initialization failed")

        print(f"Worker {worker_id}: waiting at barrier")
        barrier.wait()

        print(f"Worker {worker_id}: startup complete")
    except ValueError as e:
        print(f"Worker {worker_id}: error")
        barrier.abort()
```

```python
def safe_worker(worker_id):  1 usage
    try:
        worker(worker_id)
    except threading.BrokenBarrierError:
        print(f"Worker {worker_id}: barrier aborted, cleaning up")

if __name__ == '__main__':
    threads = [threading.Thread(target=safe_worker, args=(i,)) for i in range(NUM_WORKERS)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

See code no. 8

- BrokenBarrierErrors occur when:
  - A thread crashes
  - wait() method times out
  - When calling reset() or abort()

Semaphores

Semaphores

- Integer variable that can be incremented and decremented
- Increment and decrement operations are atomic
- Acquire operation called at the beginning of a critical section decrements integer variable
- If integer gets a negative value, acquire operation blocks
- Release operation increments integer value
- Current integer value = number of threads that can enter critical section now
- Could be greater than 1 (difference to Lock)
- Semaphore with initial value of 1 is called mutex (use Lock in this case)

Semaphores

```python
import threading
import time

connection_pool = threading.Semaphore(2)

def use_database(thread_id):  1 usage
    with connection_pool:
        print(f"Thread {thread_id}: acquired DB connection")
        time.sleep(2)
        print(f"Thread {thread_id}: released DB connection")

if __name__ == '__main__':
    threads = [threading.Thread(target=use_database, args=(i,)) for i in range(5)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

See code no. 9

Bounded Semaphores

- Semaphore allows more release operations than acquire operations
- Can be dangerous
- Bounded semaphores enforce upper limit

```python
import threading
import time

pool = threading.Semaphore(3)
# pool = threading.BoundedSemaphore(3)

def worker(worker_id):  1 usage
    pool.acquire()
    print(f"Worker {worker_id}: acquired connection")

    try:
        time.sleep(1)
    finally:
        pool.release()
        pool.release()
        print(f"Worker {worker_id}: released connection (twice)")

if __name__ == '__main__':
    threads = [threading.Thread(target=worker, args=(i,)) for i in range(5)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

See code no. 10

Events

Events

- Events are objects that are used for communication between threads
- A thread waits for a signal while another thread outputs it
- Basically, an event object manages an internal flag that can be set to true with the set() method and reset to false with the clear() method
- set() and clear() are atomic operations
- The wait() method blocks until the flag is true
- More performant than polling

Events

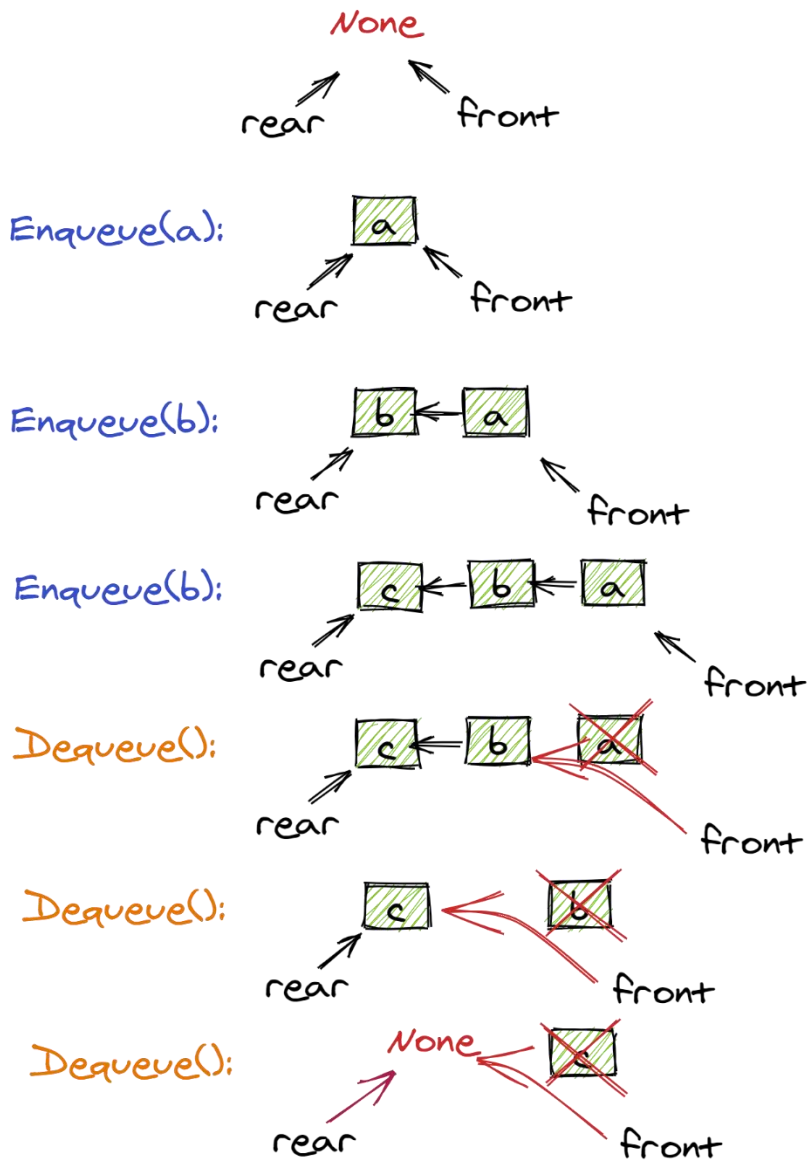
```python
import threading
import time

stop_event = threading.Event()

def worker():  1 usage
    while not stop_event.is_set():
        print("Working...")
        time.sleep(1)
    print("Stopping cleanly")

if __name__ == '__main__':
    t = threading.Thread(target=worker)
    t.start()

    time.sleep(4)
    print("Main: stopping worker")
    stop_event.set()
    t.join()
```

See code no. 11

Queues for Thread Communication

Queues for Thread Communication

- First-in-First-Out (FIFO) data structure
- Allows asynchronous N:N communication for threads
- More than two threads allowed (difference to event)
- Avoids explicit synchronization
- Preferred way of thread communication in Python



Queues for Thread Communication

Methods	Explanation
Constructor: Queue()	Creates queue
Constructor: Queue(maxsize=n)	Creates queue with maximum size of n
put(item, block=True, timeout=t)	Puts item into queue, blocks if queue is full, waits at most t seconds
get(block=True, timeout=t)	Gets item from queue, blocks if queue is empty, waits at most t seconds
task_done()	Signal that a fetched item has been fully processed, raises ValueError if called more often than get()
join()	Block until all items in the queue have been processed (queue is empty and task_done called for all extracted items)

Queues for Thread Communication

```python
import threading
import queue
import time
import random

q = queue.Queue(maxsize=5)

def producer(pid):  1 usage
    for i in range(5):
        q.put((pid, i))
        print(f"Producer {pid} -> {i}")
        time.sleep(random.randint( a: 1,  b: 3))

def consumer(cid):  1 usage
    while True:
        item = q.get()
        if item is None:
            break
        print(f"Consumer {cid} got {item}")
        q.task_done()
        time.sleep(random.randint( a: 2,  b: 5))
```

```python
if __name__ == '__main__':
    producers = [threading.Thread(target=producer, args=(i,)) for i in range(2)]
    consumers = [threading.Thread(target=consumer, args=(i,)) for i in range(3)]
    for t in producers + consumers:
        t.start()
    for t in producers:
        t.join()
    print("Wait for all items to be processed")
    q.join()
    print("Stop consumers")
    for _ in consumers:
        q.put(None)
    for t in consumers:
        t.join()
```

See code no. 12

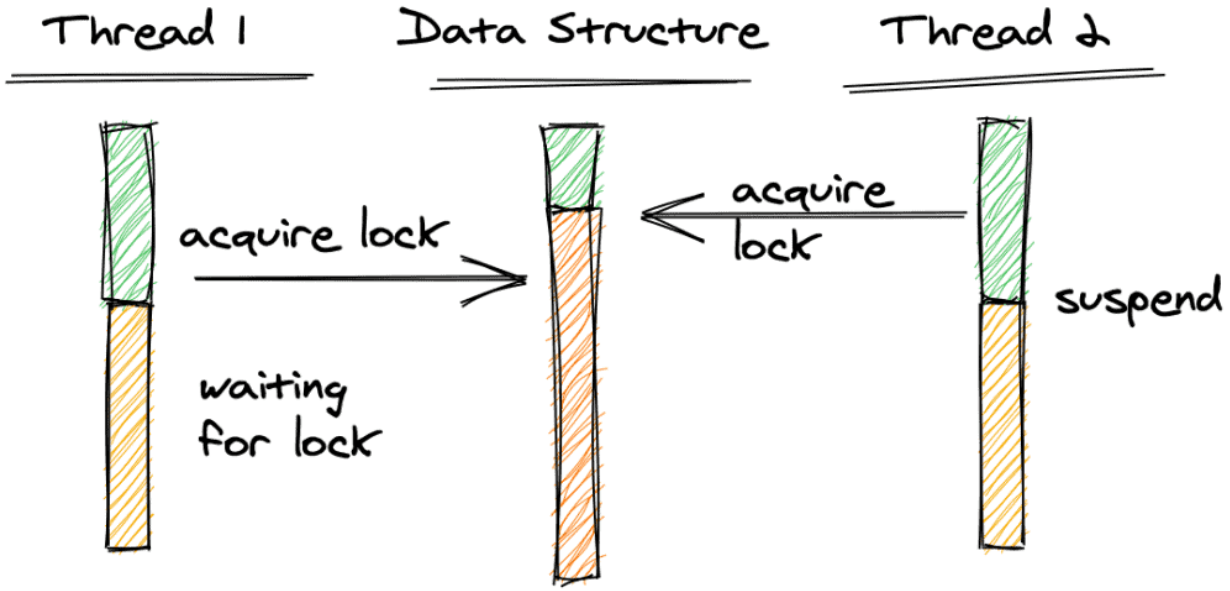
Lock-Free Data Structures

Lock-Free Data Structures

- All mechanisms shown so far use locking concept (= waiting processes are blocked)
- This is the dominant synchronization paradigm
- Alternative: Lock-Free Data Structures
  - Instead of waiting, threads retry operations if there is a conflict
  - Supported by atomic CPU operations like compare-and-swap, fetch-and-add, store-conditional
  - Atomic CPU operations are indivisible
  - Different support on different microarchitectures (ARM vs. Intel)

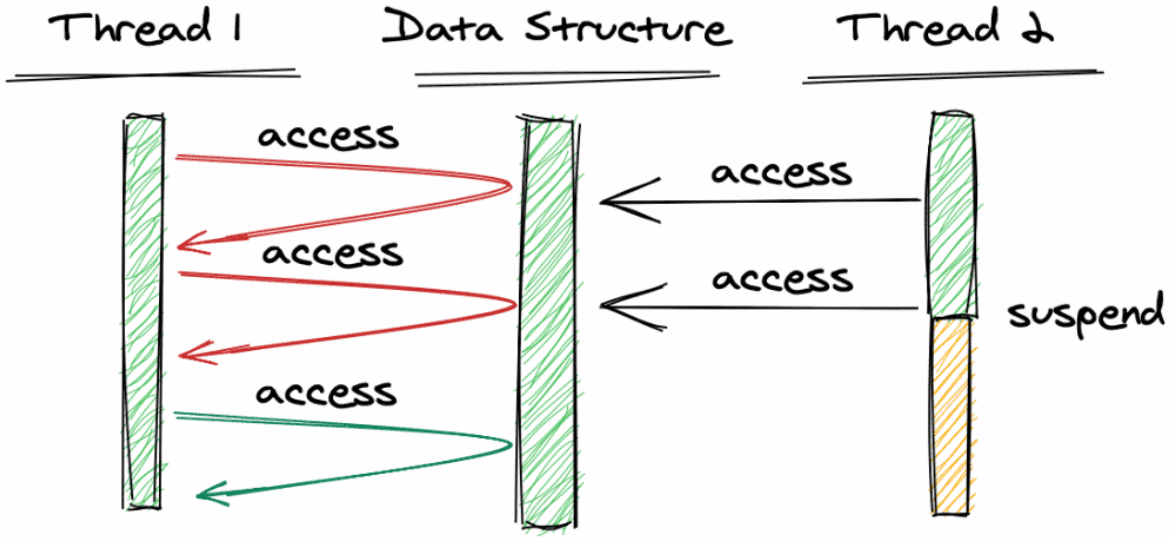
Lock-Free vs. Lock-Based

Lock-Based



One thread can block others (deadlock)

Lock-Free



No deadlocks can happen

Lock-Free Data Structures

- Disadvantages
  - Much more complex than simple locks
  - If not well implemented, can lead to race conditions and memory corruption
  - Increased resource consumption when threads are spinning
  - Performance gains only if many threads are involved
- Lock-free data structures in Python
  - Cannot be directly implemented in Python, as code is interpreted and there are no primitives to force atomic CPU operations
  - Can be realized via C code extensions (e.g., atomic_int data type)

Thread Safety and Process Safety

Thread Safety

- Code is thread-safe if it behaves correctly when **multiple threads** in the same process access it at the same time
- Examples:
  - Thread-safe mutexes / locks
  - Atomic operations
  - Thread-local storage
  - Lock-free data structures

Process Safety

- Code is process-safe if it behaves correctly when **multiple processes** access the same resource
- Examples: File locks, inter-process mutex/semaphore
- Often mediated by OS or other external entity
- Generally more expensive than thread safety

- Question: Does process safety imply thread safety?

Thread Safety != Process Safety

- A thread-safe program is not necessarily process-safe and vice versa
- Examples
  - We control the access to a file in our process with a lock to ensure that only one thread accesses to file at the same time (thread safety) -> the lock does not protect other processes to access the file
  - We control access to a file using a process-safe lock (process safety) -> multiple threads within a program may still concurrently access the file
- If you have a concurrent program with multiple processes and multiple threads, you may need both thread-safe and process-safe locks/synchronization methods
- Python: threading.Lock != multiprocessing.Lock